

Deployment-Aware Specification-Guided Repair for Ordered Guard Specifications

Ye Xujun

East China Normal University
Shanghai, China

Abstract—Ordered guard specifications enforce first-match precedence; large language models (LLMs) and test-feedback repair miss thin overlap regions that formal contracts pin down. We present SgDP, instantiated as HSP-Agile for SOFL/FSF, with three mechanisms: SpecIR first-match regions, a typed Semantic Feedback IR for repair, and a conjunctive release gate. Lead claim (C2): under equal repair budget $K=3$, full Semantic Feedback IR improves mean formal conformance by +7.7 pp vs. unstructured test-only (E6, $n=120$; paired 14/4/102; bootstrap 95% CI [2.3, 13.6] pp; Wilcoxon $p=0.018$)—scoped to test-only, not every structured trace (E14). Prevention (C3): conjunctive Accept yields E2 PDR/FAR 95.0%/5.0% vs. B2 91.2%/8.8% (bootstrap CI excludes 0). Deploy B2 by default (C4) unless Accept=1 or FAR $\leq 5\%$ is required. Equal- K M_eq reaches +2.5 pp vs. B2; industrial-pattern ($n=31$) and published-industrial ($n=10$) pilots support B2 as the mean-Conf. default. Vendor .asf1 import is supported when SpecTool/NDA packs land under vendor/.

Index Terms—specification-guided repair, ordered guards, LLM synthesis, SOFL, CEGIS, SMT, deployment boundary, industrial pilot

I. INTRODUCTION

Formal specifications buy precision that informal prose cannot match, but that precision becomes a liability when LLMs write the implementation: the model has no native notion of guard order, so it produces code that is statistically plausible yet formally wrong on thin overlap slices [1], [2]. SOFL’s Functional Specification Format (FSF) is an ordered list of scenarios under first-match precedence [3]–[5]. Unit tests almost never sample the region $G_i \wedge \bigwedge_{j < i} \neg G_j$, so test-based repair leaves ordering defects in place [6], [7]. The practical stakes are high in signalling and finance: a route-lock that ignores conflict, or a fraud-hold that loses to a transfer path, is a conformance failure even when most random tests pass.

Motivating example. Consider an industrial monitoring alert: negative sensor levels must raise Error before any threshold-based Critical path can fire. On input $(level, threshold) = (-3, -10)$, both G_1 ($level < 0$) and G_2 ($level \geq threshold$) hold; FSF mandates Error via S_1 . A typical LLM candidate evaluates the threshold comparison first and returns Critical—invisible to suites that draw non-negative thresholds. The same pattern appears in published railway SOFL cases: fault and conflict guards must preempt route-lock or gate-motion commands [8], [9]. Our pipeline turns that failure into a typed repair record naming the violated

scenario, then re-synthesises until witnesses agree or the budget is exhausted.

Research question. Can specification-conformance defects in LLM-generated code be prevented by using the specification itself as the primary evidence source, without full deductive verification? Our answer is *conditional*: yes when release requires conjunctive Accept or prevention metrics; otherwise prefer lightweight test-feedback. We therefore separate *mechanism* claims (typed feedback, Accept gate) from a *deployment* claim (when to pay for them). This separation is the difference between a tool paper that always recommends the heaviest loop and a deployment-aware method that matches cost to the release bar.

Threats this paper confronts up front. (1) Synthetic hard FSF (BENCH-120) is overlap-filtered—we bound transfer with E10, industrial-pattern, and published-industrial pilots. (2) Strengthened E1 uses $K=5$ and a multi-factor bundle—we isolate equal- K Conf. via M_eq and lead Conf. mechanism via E6. (3) No public Casco/Mitsubishi .asf1 dumps exist; we reconstruct published industrial decision tables and provide a vendor import path for SpecTool/NDA packs.

Contributions.

- **C1—SpecIR:** notation-agnostic first-match regions (engineering hinge; overlap tertiles on E3/E10 are *non-monotone*—not a ranking signal alone).
- **C2—Typed Semantic Feedback IR:** +7.7 pp vs. unstructured test-only under equal $K=3$ (E6; CI excludes 0; E14 scopes uniqueness).
- **C3—Conjunctive Accept:** full witness conformance $\wedge$ clear high-severity pattern screen (E2 PDR/FAR).
- **C4—Deployment boundary:** prefer B2 by default; escalate to M iff Accept=1 or target FAR $\leq 5\%$; pilots reinforce the default.

Paper map. Section II positions related systems; Section III details SpecIR, Feedback IR, and Accept; Section IV fixes corpora and protocol hygiene; Section V reports E6/E2/equal- K /pilots; Section VI discusses practice and threats. A companion long report carries full ablations (E14–E17), B3–B6 baselines, and figure packs; this cut keeps only numbers needed for C1–C4.

II. RELATED WORK

Closest LLM+verifier systems differ on whether witnesses respect first-match order, whether repair carries scenario structure, and whether prevention (PDR/FAR) is measured (Table I).

Proof-oriented oracles. DafnyBench and Clover target proof-oriented oracles [10], [11]; success is theorem discharge, not first-match region coverage. Our Conf. metric is witness agreement under SpecIR, not a proof certificate. CEGIS-style loops with SMT counterexamples are closer in spirit, but typically target synthesis of loop invariants or programs from logical specs rather than ordered-guard FSF with typed repair records.

Verbal and agentic repair. Self-Refine, Self-Debug, and Reflexion raise generation success via critique or verbal memory [12]–[14] but do not emit region-typed failure records. CodeT and coding agents optimise repository tasks without an FSF oracle [15]–[17]. LLM-based APR patches failing tests [18], [19]; we evaluate unstructured test feedback as B2 and isolate typed IR in E6. Break-It-Fix-It emphasises natural-language repair datasets [20]; our residual defects are ordering / overlap, not syntax.

Adjacent notations. Alloy and TLA+ encode structured constraints [21], [22]; SpecIR adapters are the intended bridge when guards are first-match and SMT-encodable. SOFL itself has a long industrial and teaching lineage [3]–[5]; we reuse that contract shape rather than inventing a new specification language.

Industrial SOFL. Railway interlocking at Casco Signal [9] and crossing controllers with Mitsubishi [8] motivate ordered-guard contracts but do not release public .asf1 dumps. Teaching cases (ATM, hotel, banking) appear in Liu’s SOFL book [4]; Agile-SOFL SpecTool demos are available only via author collaboration [3]. Related SOFL fault-prevention work studies specification-to-program transformation defects [5], [23]; we target LLM repair under an executable SpecIR oracle instead. Broader formal-methods surveys and mutation testing background [24]–[26] frame why witness-based Conf. is a pragmatic middle ground between unit tests and full proofs. SGDP combines first-match SMT witnesses, typed Semantic Feedback IR, and conjunctive Accept under PDR/FAR. We evaluate verbal/trace loops as B3–B5 and verifier-loop FSF as B6 in the long report; the conference cut focuses on B1/B2/M and E6/E2.

III. APPROACH

A. SpecIR and first-match witnesses

For task t with ordered scenarios $\langle s_1, \dots, s_n \rangle$, SpecIR exposes first-match regions as a shared hinge across notations (SOFL/FSF, Mini-Z, Mini-StateMachine adapters). Each scenario $s_i = (G_i, P_i)$ pairs a guard with a postcondition;

TABLE I
POSITIONING (TICK = DESIGN GOAL).

Approach	Spec wit.	Typed IR	Prev.	Gate
Clover / DafnyBench	partial	—	—	proof
Self-Debug / Reflexion	—	—	—	—
ChatRepair / APR	—	—	—	tests
B2 test-feedback	—	—	—	tests
B6 VerifierLoop-FSF	✓	—	—	cex
SGDP / M	✓	✓	✓	Accept

FSF selects the least i with $G_i(\mathbf{x})$ and requires $P_i(\mathbf{x}, \mathbf{y})$. For scenario s_i , the witness generator solves

$$\Phi_i \triangleq \hat{g}_i(\mathbf{x}) \wedge \bigwedge_{j < i} \neg \hat{g}_j(\mathbf{x}) \quad (1)$$

with Z3 [27], [28], yielding suite $\mathcal{D}(t)$. Without the negated higher guards, a model of g_i alone may still belong to s_j for $j < i$ —the exact failure mode of the motivating example. An *others* witness uses $\bigwedge_i \neg \hat{g}_i$ when nonempty; we disclose a prior others-witness bug (others encoded without $\neg$ prior guards) as a measurement correction that previously inflated false failures. Soundness is conditional on a faithful SMT encoding of SpecIR guards (integer/boolean fragments in our corpora). Random tests undersample Φ_i as overlap grows, so ordering bugs survive green CI [6], [7]; SpecIR makes those slices first-class for Conf. and repair.

B. Worked repair step

On $(-3, -10)$, the buggy candidate returns `Critical` where the oracle requires `Error`. The Semantic Feedback IR records (abbreviated):

```
{ "violation_type": "ordering", "scenario_index": 1,
  "constraint_text": "level < 0",
  "inputs": { "level": -3, "threshold": -10 },
  "expected": "Error", "observed": "Critical",
  "suggested_fix": "test level < 0 first" }
```

FeedbackRenderer projects this JSON into the repair prompt; E6 isolates `test_only` / `test_expected` / `semantic_ir` surfaces under identical K .

C. Semantic Feedback IR and Accept

The typed IR carries at least: `violation_type` (ordering, postcondition, others), `scenario_index`, `constraint_text`, concrete `inputs`, `expected/observed` outputs, and an optional `suggested_fix` hint. Conformance and release:

$$\text{Conf}(c, t) = \frac{1}{|\mathcal{D}(t)|} \sum_{\mathbf{x} \in \mathcal{D}(t)} \mathbf{1}[f_c(\mathbf{x}) = g_t(\mathbf{x})], \quad (2)$$

$$\text{Accept}(c, t) \iff \text{Conf}(c, t) = 1 \wedge P^H(c) = 0. \quad (3)$$

Mode B2 uses unstructured test pass/fail only—no scenario index, no constraint text. Controlled E6 isolates feedback content under full M at **equal** $K=3$ (lead Conf. claim): variant A is test-only; B adds expected outputs; C is the full Semantic IR. Equal- K mode `M_eq` ($K=3$, `semantic_ir`, advisory gate) is the Conf. ranking hygiene mode vs. B2. Strengthened E1 ($K=5$, `execution_trace_matched`, advisory gate) is a multi-factor

Algorithm 1 SgDP repair loop (HSP-Agile instantiation)

Require: FSF task t , budget K , feedback mode m

- 1: $\mathcal{D}(t) \leftarrow \text{SpecIR witnesses via Z3}$
- 2: $c^* \leftarrow \perp$; $best \leftarrow 0$
- 3: **for** $k = 1$ to K **do**
- 4: $c \leftarrow \text{synthesise / repair}(t, \text{feedback}_{k-1})$
- 5: $conf \leftarrow \text{Conf}(c, t)$ on $\mathcal{D}(t)$
- 6: **if** $conf > best$ **then** $c^* \leftarrow c$; $best \leftarrow conf$
- 7: **end if**
- 8: **if** $\text{Accept}(c, t)$ **then return** c
- 9: **end if**
- 10: $\text{feedback}_k \leftarrow \text{render}(m, \text{failures on } \mathcal{D}(t))$
- 11: **end for**
- 12: **return** c^* ▷ best-effort argmax Conf.

bundle—stress-test only, not single-factor C2. E14 further shows that length-matched execution traces can outperform *semantic_ir* on mean Conf.; uniqueness is therefore scoped to unstructured test feedback, not to “any structured string.”

D. Pipeline

Algorithm 1 summarises one task. Synthesise from FSF text $\rightarrow$ check on $\mathcal{D}(t) \rightarrow$ if not Accept, render Feedback IR and repair, up to budget K ; on exhaustion return $\arg \max_k \text{Conf}(c_k, t)$ (best-effort). Prevention evaluation injects specification-confusion and implementation-screening mutants and scores PDR/FAR on the faulty set. High-severity pattern screen P^H blocks known dangerous templates (e.g., inverted priority, missing fault preemption) even when mean Conf. is high.

Design rationale. We deliberately do *not* claim that SpecIR alone yields monotone Conf. gains with overlap density: E3/E10 tertiles are non-monotone, so overlap is an engineering hinge for witness generation, not a deployment trigger. The deployable levers are feedback typing (E6) and the Accept/prevention gate (E2). Adapters (Mini-Z, Mini-StateMachine) share SpecIR; FSF lowering is engineering for the SOFL instantiation, not a claimed contribution. The same loop applies when a partner later supplies *.asfl*: only the SpecIR front-end changes; Conf., Feedback IR, and Accept stay fixed—so vendor pilots test transfer, not a new method.

IV. EXPERIMENTAL SETUP

Benchmarks. **BENCH-120:** synthetic hard FSF (5 inputs, 8 scenarios) retained by a Z3 overlap filter; primary ranking corpus for E1/E6/E12. **E10:** removes the filter ($n=100$) to bound filter bias. **Industrial-pattern** ($n=31$): practitioner-style decision tables (banking, telecom, SCADA, ...)—not proprietary dumps. **Published-industrial pilot** ($n=10$): FSF *reconstructed* from published railway crossing / interlocking and ATM/banking SOFL cases [4], [8], [9] (benchmarks/published_industrial_pilot.json). Vendor *.asfl* files, when placed under vendor/agile-sofl-toolchain/examples, import

TABLE II
CORPUS CARD (DO NOT MIX ROWS IN ONE CLAIM).

Corpus	n	Role
BENCH-120	120	Primary E1/E6/E12 (overlap-filtered)
E10 unfiltered	100	Filter-bias bound
Industrial-pattern	31	Practitioner-style tables
Published-industrial	10	Reconstructed railway/ATM/banking
Vendor-.asfl	TBD	SpecTool/NDA import when present

via `scripts/import_vendor_asfl.py` (Casco/Mitsubishi dumps require NDA and are not redistributed).

Modes. B1 one-shot; B2 test-feedback ($K=3$); M (strengthened bundle, $K=5$); M_eq (equal- K hygiene, $K=3$, *semantic_ir*, advisory). E6 variants A/B/C hold full M fixed and vary only feedback surface.

Metrics. Mean Conf.; Strict / Accept; PDR/FAR on mutants; latency and LLM calls; paired wins/losses/ties with bootstrap CIs where claimed. Primary LLM: *ecnu-plus* (Qwen-family); secondary campus and commercial endpoints bound model dependence. Repair temperature $T_{\text{repair}}=0$. Witness budget: up to 32 formal cases per task in strengthened runs (*formal_max_cases=32*); E6 uses the same witness suite across A/B/C.

Mutants (E2). Implementation-screening mutants flip guard order, drop fault/conflict preemption, or weaken postconditions while keeping surface syntax plausible ($n=852$ screened set; pooled with specification-confusion yields $n=1704$). PDR is the fraction of faulty candidates rejected by the gate; FAR is the false-alarm rate on the same faulty pool under the release predicate.

Protocol hygiene. Cite E6 for C2; M_eq for equal- K Conf. ranking; strengthened E1 only as stress-test; never mix corpora in one claim sentence. Authoritative run ids: *run_feedback_v2* (E6), *prevention_full_v1* (E2), *run_e1_equal_k_v1*, *run_e1_m_win_v2*, *run_pubind_pilot_v1*.

V. RESULTS

A. RQ1: Typed Semantic Feedback IR (E6 / C2)

Figure 1 and Table III hold K and tasks fixed under full M ($K=3$, $n=120$). Variant C (full Semantic IR) reaches 86.9% mean Conf.—+7.7 pp over test-only (A, 79.1%) and +8.0 pp over test+expected (B, 78.9%). Paired per-task: wins/losses/ties **14/4/102**; bootstrap 95% CI [2.3, 13.6] pp; Wilcoxon $p=0.018$. Most tasks tie because they already sit near ceiling under full M; the 14 wins concentrate on ordering-sensitive items where scenario index and constraint text change the repair trajectory. E14 length-matched sweep: *semantic_ir* 75.1% vs. *execution_trace_matched* 85.4% (paired 5/19/96)—so C2 is *relative to unstructured test feedback*, not uniqueness among all structured traces. Practitioners who already emit

TABLE III
E6 PAIRED STATS AND EQUAL- K CONF. RANKING.

Contrast / mode	W/L/T or n	Conf. / Δ	95% CI
E6 C-A	14/4/102	+7.7 pp	[2.3, 13.6]
E6 C-B	14/4/102	+8.0 pp	[2.2, 14.0]
Equal- K B2	120	97.5%	—
Equal- K M_eq	120	100.0% (+2.5)	3/0/117

TABLE IV
E1 STRESS-TEST (FIXED ORACLE: $K=5$ BUNDLE FOR M).

Mode	Mean Conf.	Strict	Mean LLM calls
B1	84.2%	84.2%	1.00
B2	98.3%	98.3%	~1.18
M ($K=5$ bundle)	100.0%	100.0%	~1.16

rich execution traces may prefer those surfaces; our claim is that unstructured pass/fail is the wrong default when SpecIR witnesses are available.

B. RQ2: Equal- K ranking and E1 stress-test

Equal- K Conf. ranking (run_e1_equal_k_v1): M_eq 100.0% vs. B2 97.5% (+2.5 pp; 3/120 wins, 0 losses). This is the fair Conf. comparison at $K=3$; it is *not* a substitute for E6’s mechanism isolation. The three wins are exactly the tasks where unstructured test feedback failed to steer repair onto the first-match witness that SpecIR already knew about. Strengthened E1 bundle ($K=5$, Table IV): B1 84.2%, B2 98.3%, M 100.0% (+1.7 pp; 2/120 wins)—useful as a ceiling stress-test, not as single-factor C2. E12 (120 $\times$ 3 seeds): B2 100.0% / M 99.7% (B2 first every seed), underscoring that mean-Conf. dominance is corpus- and seed-dependent. E10 (unfiltered, $n=100$) keeps B2 near ceiling (99%) with M at 100% under the same strengthened protocol—filter removal does not invert the deployment story.

C. RQ3: Prevention (E2 / C3) and pattern gate

On implementation-screening mutants ($n=852$), M reaches PDR 95.0% / FAR 5.0% vs. B2 91.2% / 8.8%; paired bootstrap 95% CIs on the gaps exclude zero (Fig. 2). The absolute PDR lift is modest (+3.8 pp) but operationally meaningful at the FAR $\leq 5\%$ bar that many safety-adjacent release processes target. Pattern-guard value is measured here (and advisory E17), not by mean-Conf. ablation under an advisory gate (fixed-oracle A2 Δ Conf=0). Fixed-oracle ablations vs. strengthened M: removing repair -25.8 pp; removing formal checker -17.5 pp; removing pattern guard 0.0 pp on Conf. (measure gate via E2). In other words: repair and witnesses move Conf.; the pattern screen moves prevention—do not expect Conf. ablations to “prove” the gate.

D. RQ4: Deployment boundary (C4)

Canonical E3/E10 tertiles are *non-monotone*—do not escalate on overlap alone (Fig. 3). **Decision rule:** prefer B2 by default; escalate to M iff release requires Accept=1 *or* target FAR $\leq 5\%$ (Table V). On fixed-oracle E1, mean LLM calls

TABLE V
DEPLOYMENT BOUNDARY (C4).

Signal	Prefer B2	Prefer M
Default	Mean Conf./latency	—
Release	Mean Conf. OK	Accept=1
Prevention	—	FAR $\leq 5\%$
Typed repair	—	E6 +7.7 pp
Overlap	Non-monotone	Not alone

TABLE VI
PUBLISHED-INDUSTRIAL PILOT TASKS ($n=10$).

ID	Domain	Provenance
CrossingApp./Clear	Crossing	[8]
RouteLock/Signal/Point	Interlock	[9]
AtmAuth/Withdraw	ATM	[4]
HotelBook	Hotel	Liu apps
Transfer/FraudHold	Banking	Liu apps

for B2 (~1.18) and strengthened M (~1.16) are comparable; historical pre-fix Pareto exports showed larger latency gaps when repair loops ran longer—treat latency as configuration-dependent, not as a fixed 3 $\times$ tax. Fig. 5 and Fig. 6 illustrate the quality–cost trade space used to motivate the default-B2 rule.

E. External validity: industrial-pattern and published-industrial

On industrial-pattern SOFL ($n=31$), B2 reaches **100%** Conf. under gpt-4o and Claude while B1/M sit at 95.5%—strongest support for C4 default B2 on practitioner-style specs (Table VII). Published-industrial pilot ($n=10$, run_pubind_pilot_v1): B1 80.0%, B2 100.0%, M_eq 100.0% under equal $K=3$ —again B2 suffices for mean Conf.; escalate only for Accept/PDR-FAR. Table VI lists the ten reconstructed processes and their bibliographic provenance. These are reconstructions, not proprietary dumps—claims must say so.

Where vendor data comes from. Public Agile-SOFL toolchains are not on GitHub. Acquisition path: (1) Hosei/ECNU SpecTool .asfl teaching packs (sliu@hosei.ac.jp; request template in artefact vendor/REQUEST_TEMPLATE.md); (2) Casco/Mitsubishi/Nippon Signal NDA exports of process-level FSF (ordered scenarios only; strip proprietary CDFD graphics); (3) until then, published-industrial reconstructions plus optional vendor/agile-sofl-toolchain/examples import via scripts/import_vendor_asfl.py. When .asfl import succeeds, report a separate **vendor-asfl** pilot; do not relabel reconstructions as production dumps.

F. Model dependence and ceiling effects

On ecnu-max, E6 saturates ($\approx 89.2\%$ A/B/C); commercial n1n shows saturation (+0.0), modest gain (+1.0), or regression (−3.9) pp by model—typed-IR gains are headroom-dependent. Do *not* claim M dominates all seeds or corpora; lead Conf.

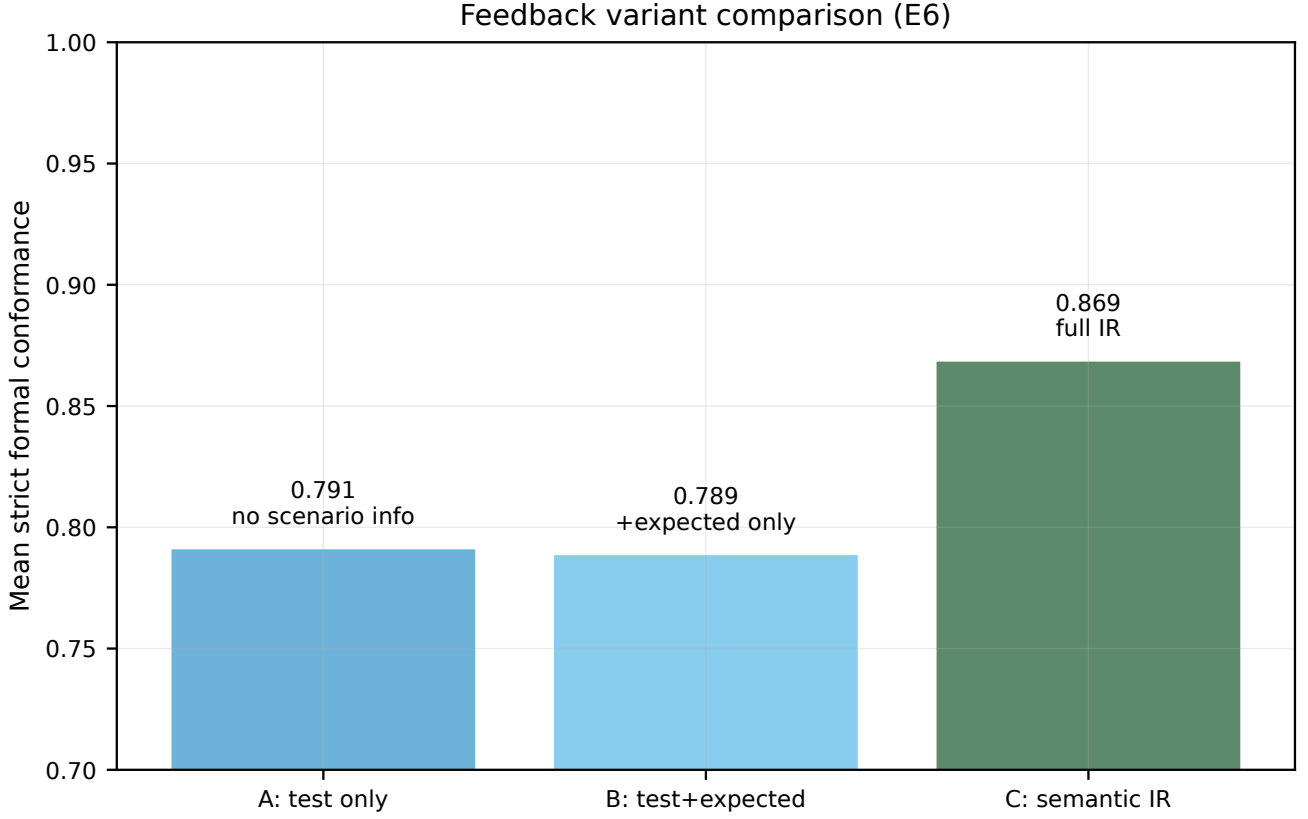


Fig. 1. E6 feedback variants under full M ($K=3$, $n=120$). Variant C (full Semantic IR) is the lead mechanism claim (+7.7 pp vs. A; CI excludes 0).

TABLE VII
EXTERNAL CORPORA (MEAN CONF. %).

Corpus	B1	B2	M / M_eq
Industrial-pattern ($n=31$, gpt-4o)	95.5	100.0	95.5
Published-industrial pilot ($n=10$)	80.0	100.0	100.0
Equal- K BENCH-120	—	97.5	100.0
E1 stress ($K=5$ M)	84.2	98.3	100.0

mechanism remains E6 under equal K on the primary LLM. Residual failures on BENCH-120 concentrate on ordering and thin-overlap slices (Fig. 9). Repair convergence under the mode-proxy curve (Fig. 10) shows most of the Conf. gain by attempt 2; typed IR (Fig. 1) is the mechanism that moves the E6 subset.

VI. DISCUSSION AND THREATS

Practice. Match feedback depth to the release bar: B2 for mean Conf. / latency; M when conjunctive Accept or $\text{FAR} \leq 5\%$ is required. Published-industrial and industrial-pattern pilots reinforce that default. For CI/CD pre-merge gates, equal- K M_eq and strengthened M have comparable mean LLM calls to B2 on fixed-oracle E1; interactive IDE use may still prefer B2. Railway-style fault/conflict preemption is exactly where Accept and PDR/FAR matter more than mean Conf.: a candidate that scores high mean Conf. but misses a fault-first scenario must

not ship. A concrete playbook: (i) run B2 in the inner loop; (ii) nightly or pre-release run M with Accept; (iii) track FAR on a frozen mutant pack; (iv) escalate the inner loop only if FAR drifts above target or Accept failures cluster on ordering classes.

Published pilot vs. vendor-.asf1. The $n=10$ published-industrial pilot is intentionally honest: guards are reconstructed from papers and textbooks, not extracted from Casco production databases. That is enough to stress first-match preemption on railway/ATM/banking shapes and to support C4’s default-B2 recommendation on mean Conf. It is *not* a substitute for a partner NDA dump. When SpecTool or NDA .asf1 files arrive under vendor/agile-sofl-toolchain/examples, re-export with `import_vendor_asf1.py` and report a separate vendor-asf1 table—same metrics, clearer provenance.

What not to claim. Do not claim (i) M dominates every seed (E12), (ii) overlap tertiles predict when to escalate (E3/E10), (iii) Semantic IR is unique among all structured traces (E14), or (iv) published reconstructions are Casco production dumps. Lead story remains E6 mechanism + E2 prevention + C4 deployment.

Threats to validity. (1) *Construct*: finite witnesses approximate Conf.; others-witness bug disclosed as measurement correction that previously inflated false failures (~5% Strict on historical E1). (2) *Internal*: LLM non-determinism mit-

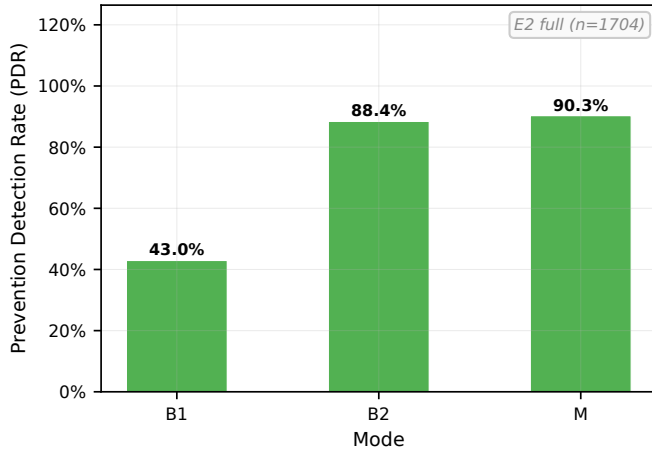


Fig. 2. E2 aggregate PDR by mode.

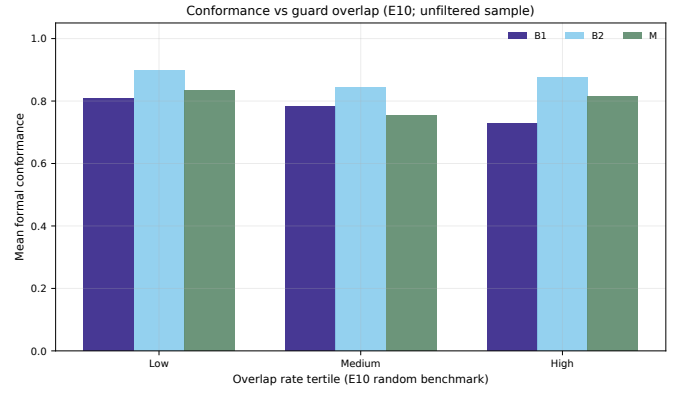


Fig. 3. E10 Conf. vs. overlap tertile (non-monotone).

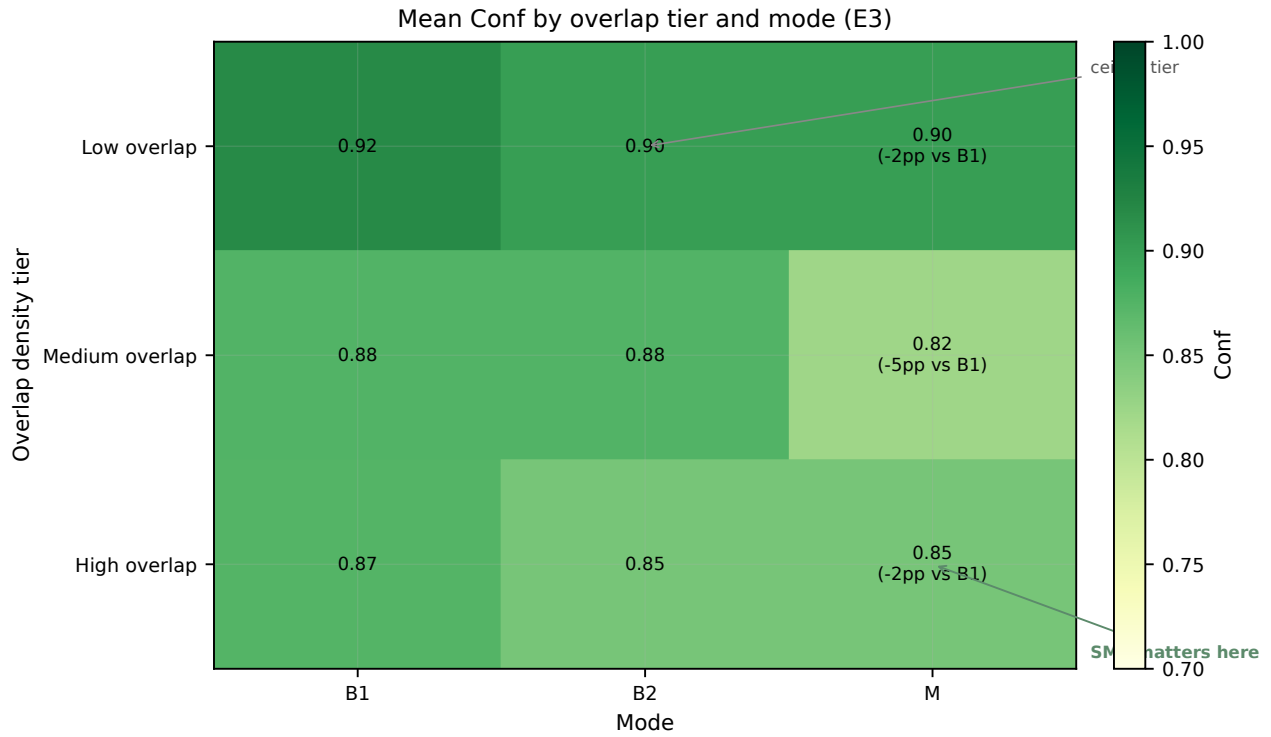


Fig. 4. Complexity / density heatmap (companion view; not a deployment trigger).

igated by $T_{\text{repair}}=0$ and paired tests; equal- K hygiene via M_{eq} ; E1 bundle is not E6. (3) *External*: BENCH-120 is synthetic/overlap-filtered; E10, E11, industrial-pattern, and published-industrial pilots bound transfer. Reconstructions are not proprietary dumps. Vendor .asf1 pilots remain future work until SpecTool/NDA packs are obtained via the Hosei contact path or partner NDA. (4) *Conclusion*: do not read the $K=5$ E1 bundle as equal- K C2; power for tiny Conf. gaps is low, hence paired win rates and mechanism CIs. (5) *Model*: ceiling effects on stronger models shrink E6 deltas—report headroom explicitly when transferring claims.

Ethics and data. Industrial NDA artefacts must not be redistributed; published reconstructions cite bibliographic provenance only. Contact templates for SpecTool packs live in the artefact repository under `vendor/REQUEST_TEMPLATE.md` (Hosei starting point: `sliu@hosei.ac.jp`). Aggregate Conf./PDR only when partners forbid process-name disclosure. Reproducibility artefacts (run ids, benchmark JSON, import scripts) accompany the long report; this cut cites the same authoritative numbers card.

Reproducibility. Benchmarks, run ids, and summarisation scripts ship with the artefact package:

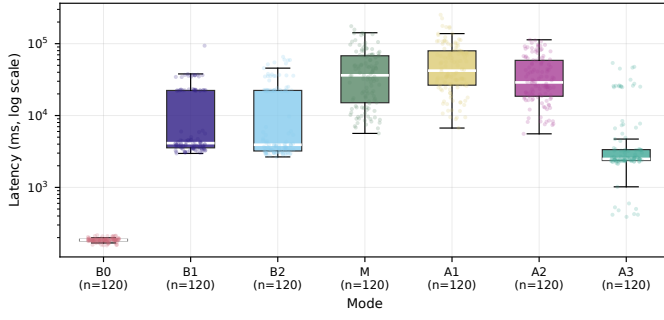


Fig. 5. Latency distribution by mode (log scale; configuration-dependent).

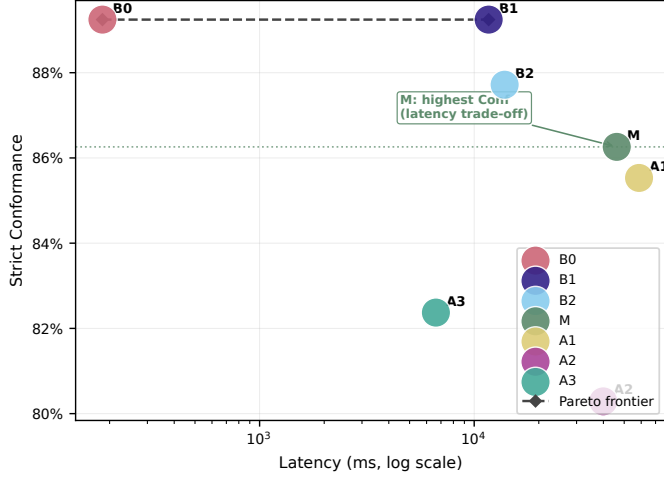


Fig. 6. Quality vs. latency Pareto view motivating default B2 for mean Conf.

benchmarks/published_industrial_pilot.json, scripts/import_vendor_asfl.py, paper/hsp-agile/scripts/e6_paired_analysis.py, and AUTHORITATIVE_NUMBERS.md. Primary claims must cite those run directories rather than regenerated ad-hoc aggregates. A long-report companion expands B3–B6, E14–E17, and full figure packs for readers who need the complete experimental appendix.

VII. CONCLUSION

SgDP/HSP-Agile prevents first-match ordering defects with SpecIR witnesses, typed Semantic Feedback IR, and conjunctive Accept. Remember: (1) E6 +7.7 pp vs. test-only at $K=3$ (CI excludes 0; E14 scopes uniqueness); (2) E2 PDR/FAR justifies the gate; (3) deploy B2 by default; escalate for Accept or $\text{FAR} \leq 5\%$; (4) equal- K , pilots, and ceiling stress-tests are secondary to the mechanism+deployment story. On published-industrial and industrial-pattern corpora, mean Conf. already favours B2—use M when the release bar demands Accept or prevention, not by default. Future work: NDA vendor .asfl pilots at scale, adaptive K , and SpecIR adapters beyond SOFL/FSF.

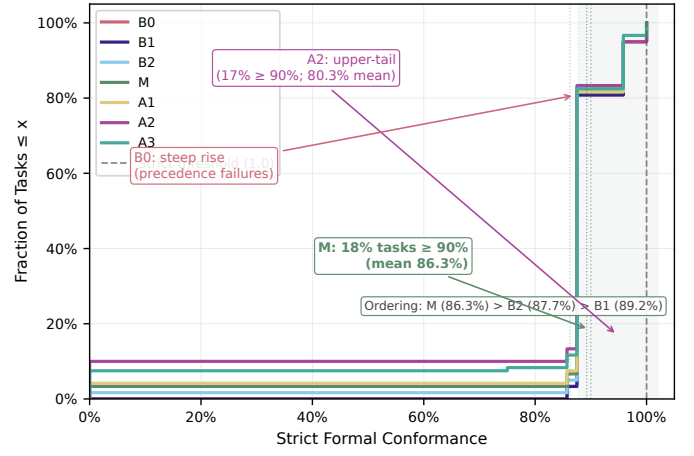


Fig. 7. Formal conformance CDF (E1 stress-test context).

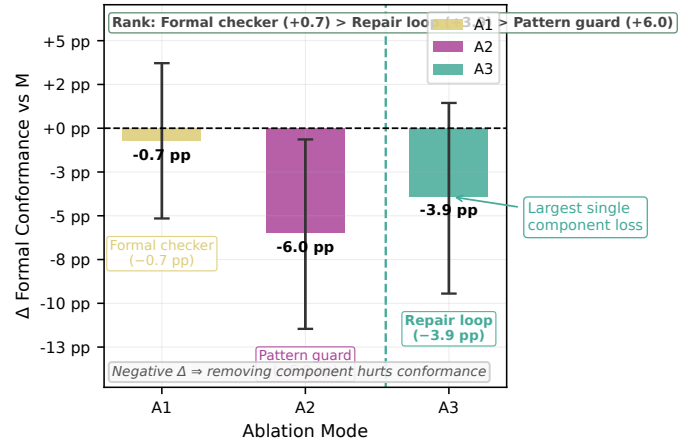


Fig. 8. Fixed-oracle ablation $\Delta\text{Conf.}$ vs. strengthened M.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [2] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. Jiang, “GitHub Copilot AI pair programmer: Asset or liability?” *Journal of Systems and Software*, vol. 203, p. 111734, 2023.
- [3] S. Liu, A. J. Offutt, C. Ho-Stuart, Y. Sun, and M. Ohba, “SOFL: A formal engineering methodology for industrial applications,” *IEEE Transactions on Software Engineering*, vol. 24, no. 1, pp. 24–45, 1998.
- [4] S. Liu, *Formal Engineering for Industrial Software Development: Using the SOFL Method*. Berlin, Heidelberg: Springer, 2004.
- [5] J. Li and S. Liu, “Requirements-related fault prevention mechanism for SOFL formal specification-based programming,” in *Proc. 22nd IEEE Int. Conf. Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, 2022, pp. 359–367.
- [6] H. Zhu, P. A. V. Hall, and J. H. R. May, “Software unit test coverage and adequacy,” *ACM Computing Surveys*, vol. 29, no. 4, pp. 366–427, 1997.
- [7] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge University Press, 2016.
- [8] S. Liu, M. Asuka, K. Komaya, and Y. Nakamura, “Applying SOFL to specify a railway crossing controller for industry,” in *Proceedings of the 2nd IEEE Workshop on Industrial Strength Formal Specification Techniques*, 1998, pp. 16–27.

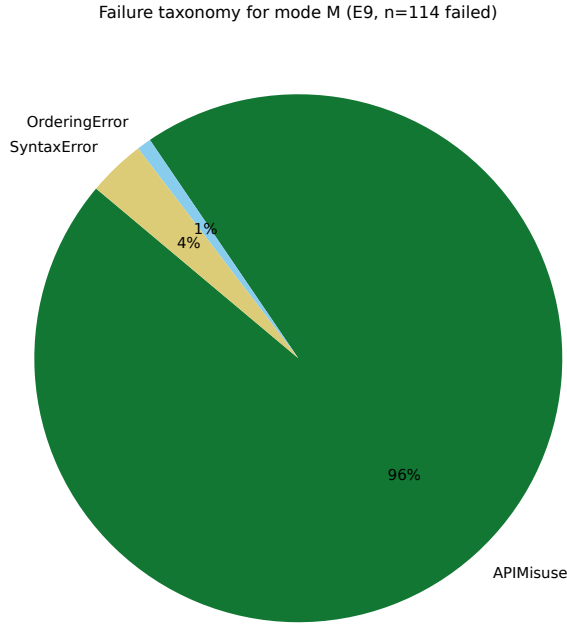


Fig. 9. Residual failure taxonomy (ordering / overlap dominate).

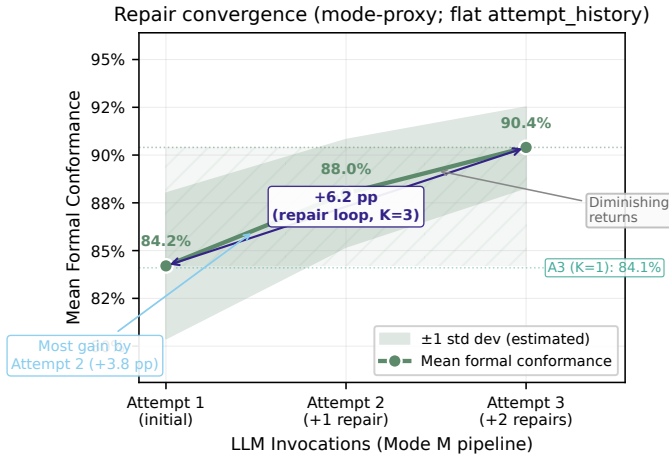


Fig. 10. Repair convergence across attempts (mode-proxy; archived attempt-history is flat under the fixed oracle).

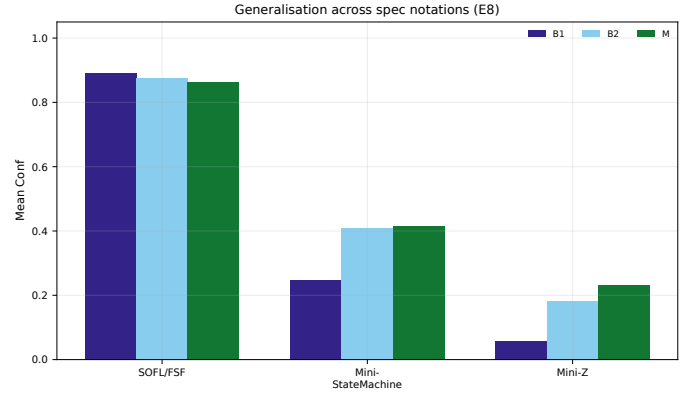


Fig. 11. Cross-setting generalisation bars (companion to industrial pilots).

- [9] J. Luo, S. Liu, Y. Wang, and T. Zhou, "Applying SOFL to a railway interlocking system in industry," in *Structured Object-Oriented Formal Language and Method (SOFL+MSVL)*, ser. LNCS, vol. 10189. Springer, 2017, pp. 160–177.
- [10] J. Loughman and T. Ringer, "DafnyBench: A benchmark for formal software verification," *arXiv preprint arXiv:2406.08467*, 2024.
- [11] C. Sun, Y. Sheng, O. Padon, and C. Barrett, "Clover: Closed-loop verifiable code generation," in *Proc. 1st Int. Symp. AI Verification (SAIV 2024)*, ser. Lecture Notes in Computer Science, 2024.
- [12] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [13] X. Chen *et al.*, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.
- [14] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] B. Chen *et al.*, "CodeT: Code generation with generated tests," in *International Conference on Learning Representations (ICLR)*, 2023.
- [16] X. Wang *et al.*, "OpenDevin: An Open Platform for AI Software Developers as Generalist Agents," *arXiv preprint arXiv:2407.16741*, 2024.
- [17] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," *arXiv preprint arXiv:2405.15793*, 2024.
- [18] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, chatRepair; LLM-based APR.
- [19] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023.
- [20] M. Yasunaga and P. Liang, "Break-it-fix-it: Unsupervised learning for program repair," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, 2021, pp. 11 941–11 952.
- [21] D. Jackson, "Alloy: A lightweight object modelling notation," *ACM Transactions on Software Engineering and Methodology*, vol. 11, no. 2, pp. 256–290, 2002.
- [22] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Boston, MA: Addison-Wesley, 2002.
- [23] J. Li and S. Liu, "Requirements-related fault prevention during the transformation from formal specifications to programs," *IET Software*, vol. 17, no. 3, pp. 316–332, 2023.
- [24] J. Woodcock, P. G. Larsen, J. Bicarregui, and J. Fitzgerald, "Formal methods: Practice and experience," *ACM Computing Surveys*, vol. 41, no. 4, pp. 1–36, 2009.
- [25] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.
- [26] K. R. M. Leino, "Dafny: An automatic program verifier for functional correctness," in *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR)*, ser. Lecture Notes in Computer Science, vol. 6355. Springer, 2010, pp. 348–370.
- [27] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.
- [28] C. Barrett and C. Tinelli, "Satisfiability modulo theories," *Handbook of Model Checking*, pp. 305–343, 2018.

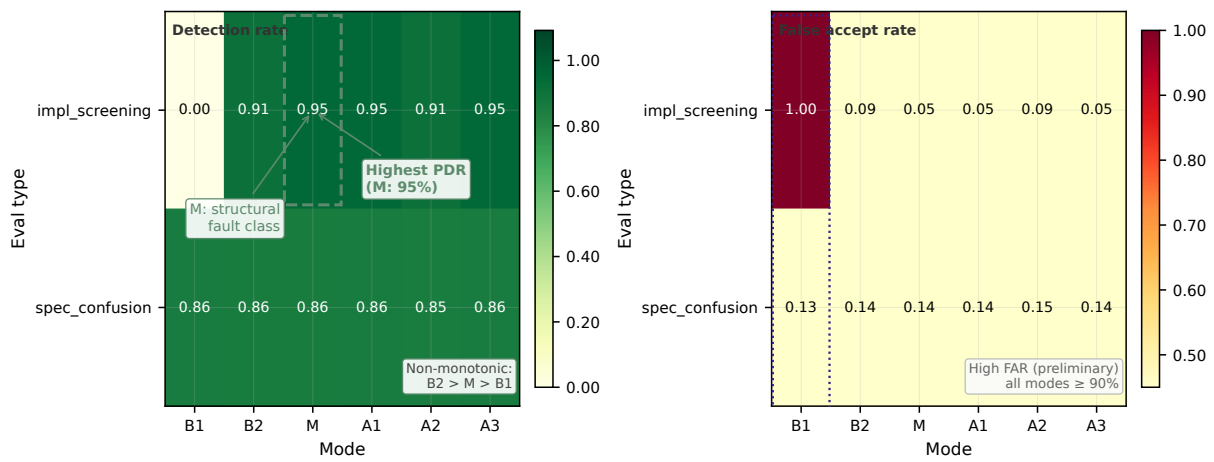


Fig. 12. E2 prevention heatmap by mode and evaluation slice.

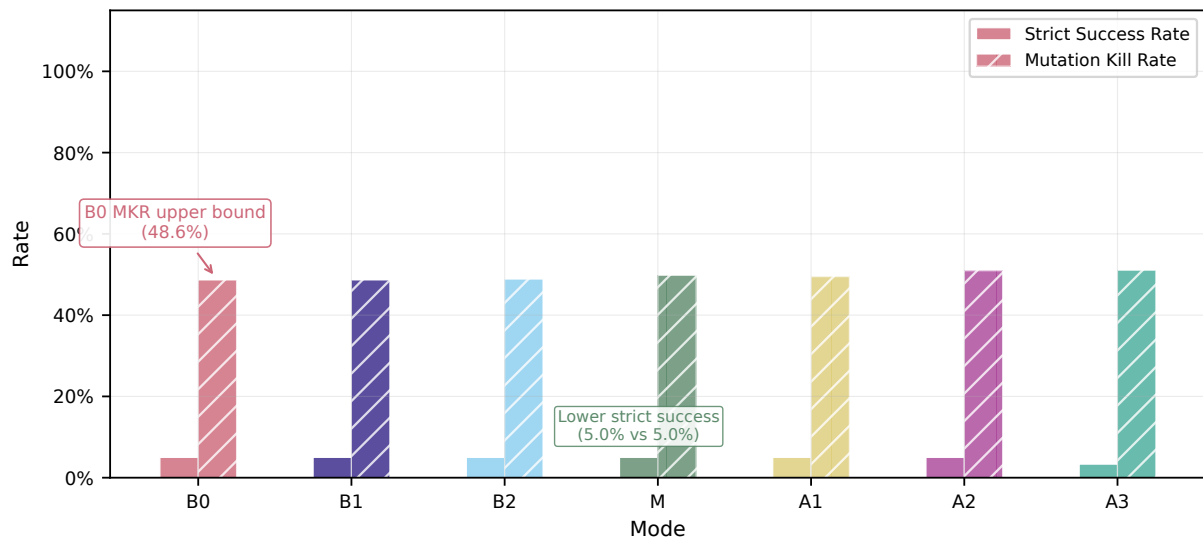


Fig. 13. Summary metrics by mode (E1 stress-test context; cite equal- K and E6 for lead Conf. claims).